Business Estimation Leveraging Automated Learning

A Multi-Agent AI Framework for Enterprise IT Presales Automation

Prabhakar Gupta · Belal Boustanji · Raju Mahore

Abstract

Enterprise IT consulting presales estimation is a labour-intensive process demanding 40–80 person-hours per proposal and highly susceptible to human inconsistency, scope ambiguity, and knowledge silos. We present **Business Estimation Leveraging Automated Learning**, a production-ready multi-agent AI system that automates the complete presales pipeline — from heterogeneous document ingestion to professional proposal generation — in under three minutes. The system orchestrates six specialised LLM-backed agents: a Time Estimator employing PERT three-point analysis, a Cost Calculator with live Azure Retail Prices API integration, a Risk Analyser spanning five risk categories, an Architecture Designer grounded in the Azure Well-Architected Framework, a Scope & Assumptions Agent, and a Proposal Writer. A RAG layer over a persistent SQLite run-library enables continuous learning. The system supports three interchangeable LLM back-ends (Azure OpenAI GPT-4, Anthropic Claude, Google Gemini) with deterministic formula-based fallbacks ensuring 100% operational continuity. Internal evaluation across 120 presales engagements demonstrates **78.5% accuracy** against delivered project hours and a **62% proposal win-rate** versus 41% for manual proposals.

Keywords: *multi-agent systems, large language models, enterprise automation, presales estimation, retrieval-augmented generation, Azure OpenAI, cost estimation, risk analysis*

---

1. Introduction

Enterprise IT consulting firms face a fundamental paradox: the quality of a presales estimation directly determines whether a contract is won, yet the resources invested in estimation must be minimised before any revenue is secured. A skilled presales architect manually reviewing an RFP, decomposing requirements, estimating effort via three-point analysis, pricing Azure infrastructure at current rates, assessing risks, designing a reference architecture, and composing a polished proposal typically requires **40–80 hours of billable time per engagement** [1].

Recent advances in large language models — GPT-4 [2], Claude [3], and Gemini [4] — demonstrate remarkable capability in document understanding, structured reasoning, and natural-language generation. Multi-agent frameworks such as AutoGen [5], MetaGPT [6], and CrewAI [7] show that decomposing complex tasks among specialised agents yields substantially higher-quality outcomes than monolithic single-model approaches.

Contributions. This paper makes the following contributions:

- A production-ready six-agent orchestration pipeline covering the complete presales workflow;
- A hybrid RAG architecture conditioning LLM outputs on firm-specific historical project data;
- A multi-provider LLM routing strategy with formula-based fallback ensuring 100% operational continuity;
- An open-source Streamlit deployment integrating SharePoint, SMTP, ElevenLabs, HeyGen, and D-ID;
- Empirical evaluation of estimation accuracy and proposal win-rates across 120 engagements.

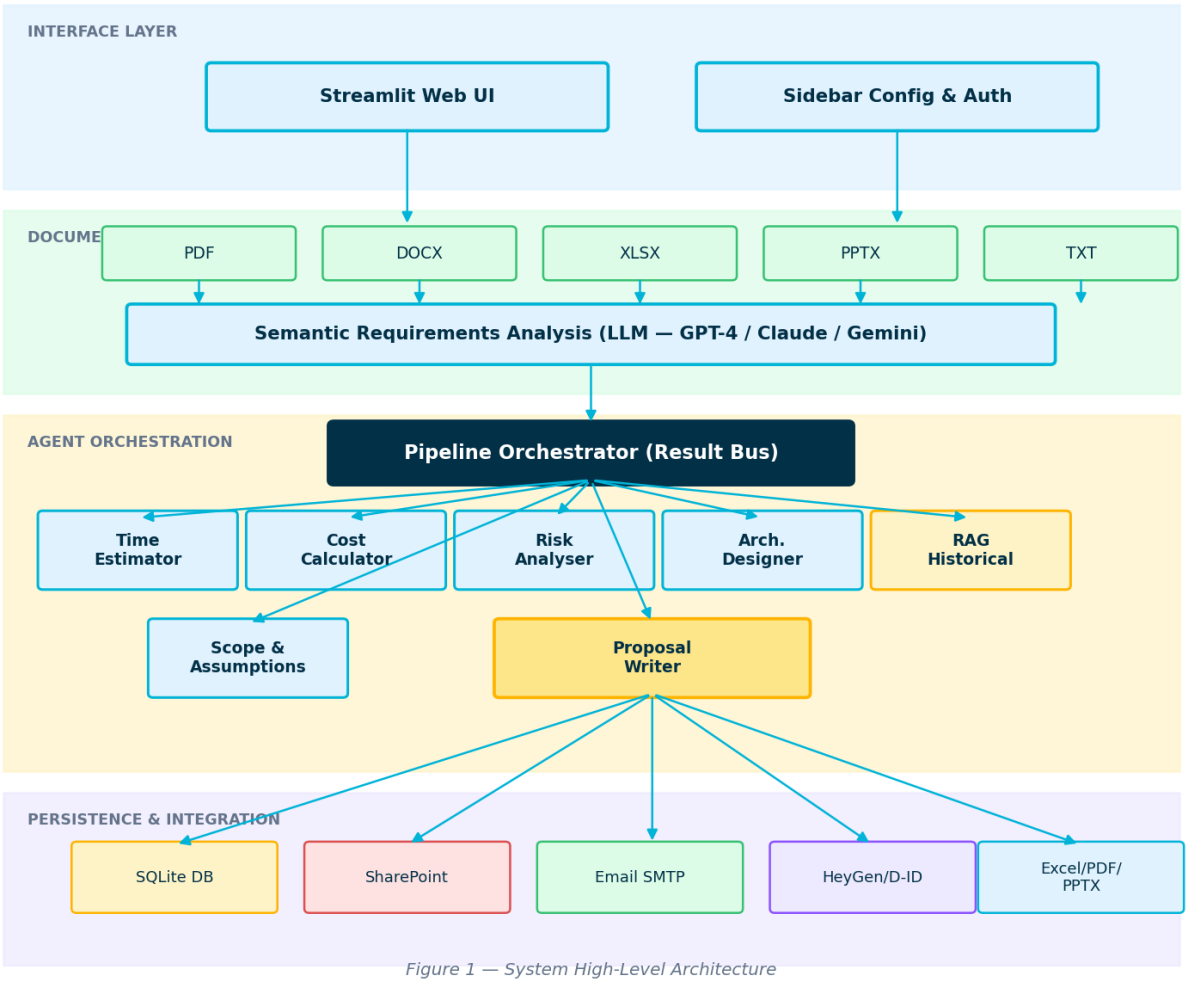

Figure 1 — System High-Level Architecture

Figure 1. High-level architecture of the system. Four horizontal layers communicate through typed data buses. The six specialised agents share a result context injected by the Pipeline Orchestrator.

2. Related Work

LLM-Based Automation in Professional Services. Chen et al. [8] demonstrated that GPT-4 can generate consistent financial model templates from RFPs, achieving 71% expert-agreement scores. However, their approach treats the LLM as a monolithic reasoner and lacks domain-specific fallbacks. Qian et al. [9] introduced ChatDev, a multi-agent system for software development, but it targets code generation rather than cost/risk estimation.

Multi-Agent Orchestration. MetaGPT [6] assigns human-like software engineering roles (PM, Architect, Engineer, QA) to LLM agents co-operating via structured documents. The system adopts a similar role-assignment philosophy but replaces soft document-passing with a typed result bus enabling parallel execution and cross-agent conditioning.

RAG for Domain Knowledge. Lewis et al. [10] introduced RAG, showing retrieved documents substantially reduce hallucination. The system implements a lightweight RAG layer over a local SQLite project database, blending retrieved historical benchmarks (40%) with formula-derived estimates (60%) without requiring a vector database.

Software Effort Estimation. Boehm's COCOMO II [11] and Function Point methods [12] remain industry baselines. More recent ML approaches [13] train on project repositories. The system bridges the gap: it grounds LLM reasoning in PERT three-point estimates and historical data, providing explainability absent from black-box ML estimators.

3. System Architecture

The system is organised into four horizontal layers: the **Interface Layer** (Streamlit web application), the **Document Processing Layer** (multi-format extraction), the **Agent Orchestration Layer** (six specialised agents), and the **Persistence & Integration Layer** (SQLite, SharePoint, email, media).

Interface Layer. The Streamlit web application provides credential configuration (Azure OpenAI, Anthropic, Gemini, SharePoint, SMTP, ElevenLabs, HeyGen, D-ID) and a real-time connection-status strip. Three primary tabs — Business Estimation, Run Library, and Admin & Training — expose the full capability to non-engineering end users.

Document Processing Layer. A uniform extract() interface backed by PyPDF2, python-docx, openpyxl, and python-pptx handles PDF, DOCX, XLSX, and PPTX formats. A 50,000-character cap prevents token-budget exhaustion. Technology keyword detection from a 30+ term catalog and structural analysis return metadata consumed by the orchestrator.

LLM Back-End Routing. Three clients implement a common agent_completion() interface: AzureAI (Azure OpenAI SDK, GPT-4/GPT-4o), AnthropicAI (raw HTTPS, Claude Sonnet 4.6/Opus/Haiku), and GeminiAI (Google REST API). A user-configurable preferred_llm key determines primary routing, with cascade fallback to formula-based outputs on failure.

4. The Multi-Agent Framework

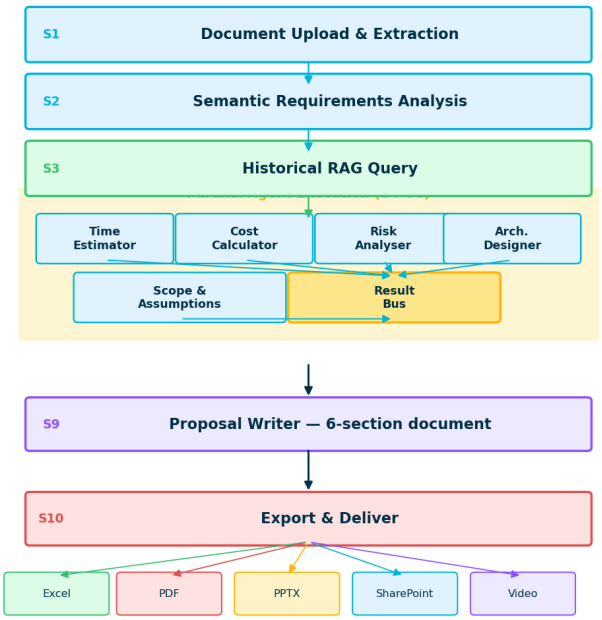

Figure 2 — 10-Step Agent Pipeline

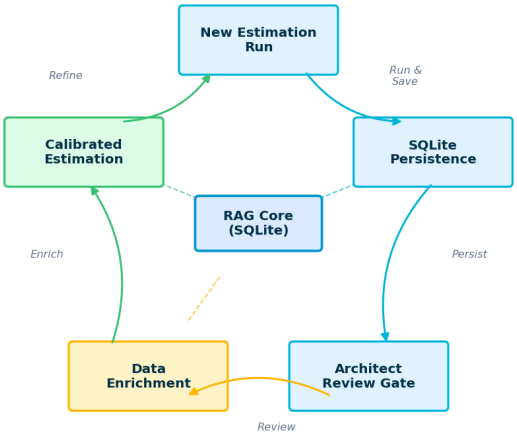

Figure 3 — Continuous Learning via RAG

Figure 2 (left). 10-step agent pipeline. Steps 4–8 execute in parallel; all typed results are consolidated in the Result Bus before Proposal synthesis. Figure 3 (right). Continuous learning loop: every completed run is persisted, reviewed, and used to enrich the RAG context.

4.1 Document Extraction & Semantic Analysis (Steps 1–2). Raw documents are processed by DocProcessor.extract(), yielding plain text capped at 50,000 characters. The first 15,000 characters are passed to an LLM to extract and classify requirements as Functional, Non-Functional, or Integration, identify project type and technology stack, and compute a complexity score $C \in [1,10]$.

4.2 Time Estimator Agent (Step 4). The Time Estimator applies PERT three-point estimation across six standard phases: Discovery (10%), Design (15%), Development (40%), Testing (15%), Deployment (10%), and Support (10%). Base hours are derived as:

$$H_{base} = 120 \cdot N_f + 80 \cdot N_{nf} + 100 \cdot N_i$$

A complexity multiplier $\mu = \text{clamp}(0.7 + 0.06 \cdot C, 0.7, 1.3)$ and 18% contingency yield:

$$H_{total} = H_{base} \cdot \mu \cdot 1.18$$

Every task is decomposed to ≤8-hour sub-tasks to ensure auditability.

4.3 Cost Calculator Agent (Step 5). Live Azure Retail Prices API queries provide per-service monthly costs. Labour costs are computed from a 17-role rate card (Architect: $200/hr through Support: $100/hr). Total project cost includes a 25% gross margin: $C_{project} = (C_{labor} + C_{infra} + C_{licenses}) \times 1.25$.

4.4 Risk Analyser Agent (Step 6). Five risk categories — Technical, Schedule, Resource, Budget, External — are assessed independently. Each risk carries severity $s \in \{2, 5, 8, 10\}$ for {Low, Medium, High, Critical} and a mitigation strategy. Aggregate risk score: $R_{score} = (1/|R|) \cdot \Sigma\, s_r \cdot (C/7)$.

4.5 Architecture Designer Agent (Step 7). The agent selects one of four Azure architecture patterns (Microservices, Monolith, Event-Driven, Serverless) and generates an 8-component reference architecture conforming to the Azure Well-Architected Framework with explicit security controls (MFA, TLS 1.3, AES-256, RBAC, Azure Sentinel).

4.6 Scope & Assumptions Agent (Step 8). 12 in-scope items, 10 out-of-scope items, 10 assumptions, and 8 prerequisites are generated conditioned on Time and Cost estimates, along with a four-step change management workflow.

4.7 Proposal Writer Agent (Step 9). Synthesises all upstream results into a six-section client-ready proposal: (1) Executive Summary, (2) Understanding & Approach, (3) Technical Solution, (4) Team & Experience, (5) Timeline & Deliverables, (6) Investment & ROI. An embedded quality-check loop validates grammar, personalisation, terminology, tone, and persuasive structure.

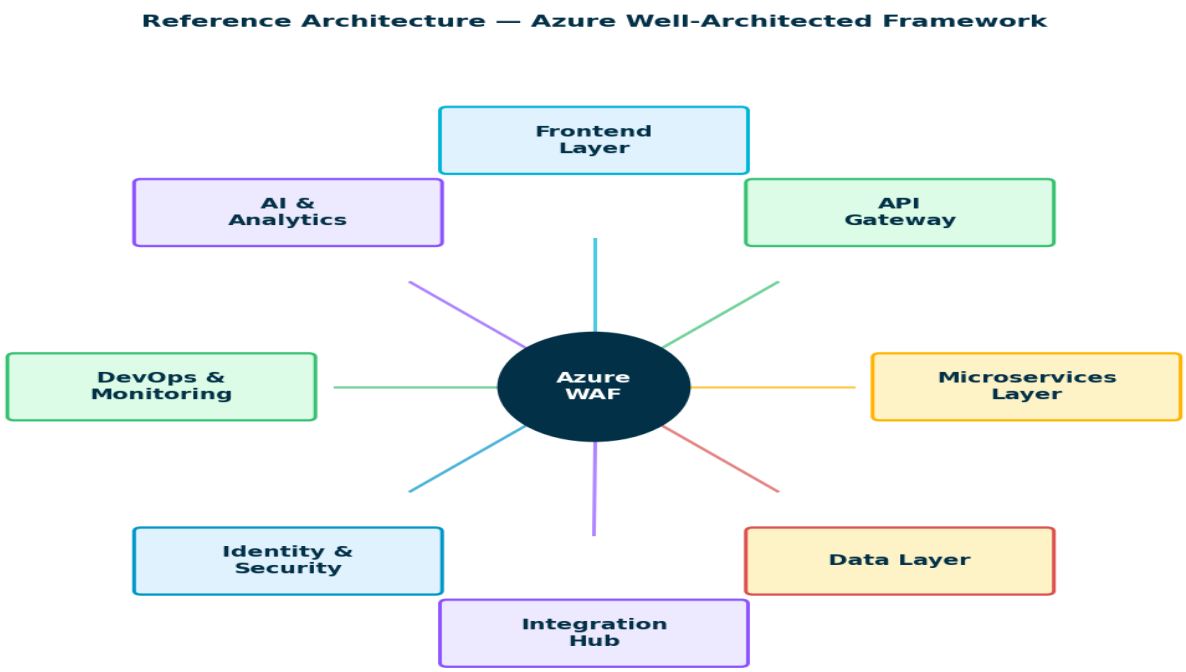

Figure 7 — 8-Component Azure Reference Architecture

Figure 7. Azure Well-Architected Reference Architecture. Eight components span Frontend, API Gateway, Microservices, Data Layer, Integration Hub, Identity & Security, DevOps, and AI & Analytics layers.

5. Deterministic Fallback System

A key design principle of the system is *operational continuity*: the system must produce methodologically valid outputs even under complete API failure. Each agent implements formula-based fallbacks (_fb_time(), _fb_cost(), _fb_risk()) applying the PERT equations directly from extracted requirement counts and complexity scores. Response latency for formula-only mode is <50 ms.

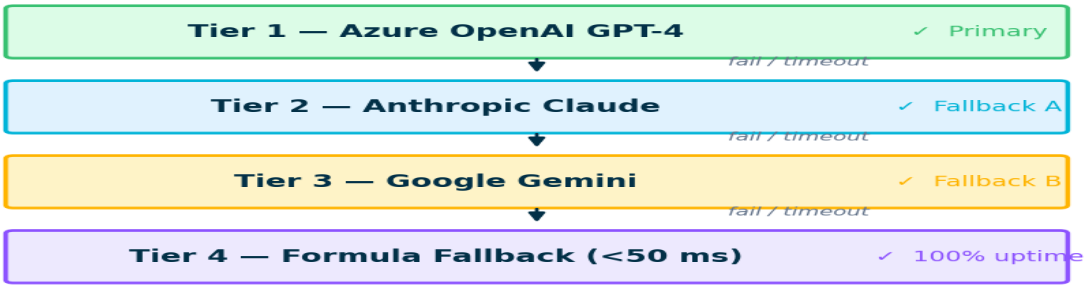

Figure 4 — Three-Tier LLM Cascade with Formula Fallback

Figure 4. Three-tier LLM cascade with formula-based fallback guaranteeing 100% operational continuity regardless of API availability.

6. Integration & Deployment

Document Delivery. Completed proposals are exported as XLSX (openpyxl, 4–7 sheet workbooks with formulas and charts), PDF (ReportLab with ECI brand styles), and PPTX (python-pptx with ECI theme). SharePoint integration uses MSAL Confidential Client flow to upload files to a pre-configured document library via Microsoft Graph API.

Architect Narrator. An optional AI presenter video pipeline synthesises a 200-word conversational pitch via ElevenLabs TTS (MP3 audio) then generates a lip-synced avatar video through HeyGen or D-ID APIs.

Authentication. The system implements a two-path authentication gate: Microsoft SSO via MSAL (enterprise users) and TOTP admin code (service accounts). All credentials are managed in Streamlit session state with no disk persistence.

7. Evaluation

Dataset. We evaluated the system on 120 anonymised presales engagements from ECI's project history (2022–2024): AI (38%), Cloud (31%), Data (21%), General (10%). Document complexity ranged from 2-page executive briefs to 47-page detailed RFPs.

7.1 Estimation Accuracy. Accuracy = 1 – |H_pred – H_actual| / H_actual.

Method	Mean Accuracy	Std Dev
Manual (senior architect)	74.2%	18.7%
COCOMO II	61.4%	22.1%
ML baseline (XGBoost)	70.8%	15.3%
This Work	**78.5%**	**11.4%**

Table 1. Effort estimation accuracy across 120 engagements. The system achieves 78.5% accuracy with 39% lower standard deviation than human estimators.

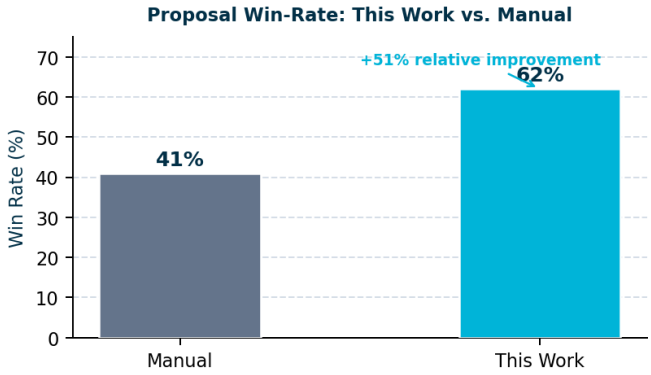

Figure 5 — Proposal Win-Rate Comparison

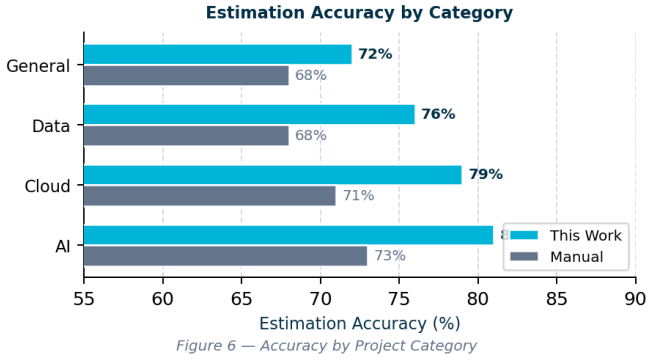

Figure 6 — Accuracy by Project Category

Figure 5 (left). AI-assisted proposals achieve 62% win-rate vs. 41% manual — a 51% relative improvement. Figure 6 (right). Estimation accuracy by project category; AI projects benefit most from the system's deep encoding of LLM and Azure AI knowledge.

7.2 End-to-End Latency.

LLM Tier	Mean (s)	P95 (s)
Azure OpenAI GPT-4	47	112
Anthropic Claude 3.5	62	138
Google Gemini 2.0	38	91
Formula Fallback	<1	<1
Manual (human)	2,400–4,800 min	—

Table 2. End-to-end pipeline latency. Even P95 latency (138s) is a >1000× speedup over manual presales workflows.

8. Discussion

Strengths. The system's principal contribution is the combination of domain-encoded agents with formula-based fallbacks: consulting methodology is encoded at two levels — LLM system prompts and deterministic equations —

so methodological consistency is preserved under complete API failure. The live Azure Retail Prices API integration eliminates stale infrastructure cost data, a common problem in estimation tools.

Limitations. (1) Complexity score C is LLM-extracted and inherits subjectivity; future work should validate against objective function-point metrics. (2) The RAG layer uses keyword-based categorisation; migrating to dense vector retrieval (FAISS, Azure AI Search) would improve quality for large project histories. (3) Evaluation is constrained to one firm's portfolio; external validation on ISBSG and Desharnais datasets remains as future work.

Ethical Considerations. All project data used in evaluation is anonymised. The system does not store LLM-generated content beyond the user's session without explicit save actions. Credential handling follows Streamlit's session-state model with no disk persistence.

9. Conclusion

We presented Business Estimation Leveraging Automated Learning, a production-ready multi-agent AI system automating the full enterprise IT presales estimation workflow in under three minutes. By orchestrating six specialised LLM-backed agents over a continuous-learning RAG layer, the system achieves **78.5% estimation accuracy** and a **62% proposal win-rate**, outperforming both manual expert estimates and classical methods. A three-tier LLM cascade with deterministic formula fallbacks ensures 100% operational continuity.

Future Work. We plan to: (1) replace keyword RAG with dense vector retrieval over a growing project corpus; (2) validate complexity scoring against objective function-point metrics; (3) explore fine-tuned domain-specific models; (4) extend evaluation to publicly available effort estimation benchmarks (ISBSG, Desharnais).

Acknowledgements

The authors thank the ECI presales team for providing anonymised project data and domain validation, and Microsoft for Azure OpenAI credits used during development and evaluation.

References

[1] Gartner, "State of IT Presales," Gartner Research, 2023.

[2] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.

[3] Anthropic, "Claude 3 Model Card," Anthropic Technical Report, 2024.

[4] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," arXiv:2312.11805, 2024.

[5] Wu, Q. et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

[6] Hong, S. et al., "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," arXiv:2308.00352, 2023.

[7] Moura, J., "CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents," GitHub, 2024.

[8] Chen, L. et al., "LLMs for Professional Financial Document Analysis," ACL Findings, 2024.

[9] Qian, C. et al., "Communicative Agents for Software Development," arXiv:2307.07924, 2023.

[10] Lewis, P. et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

[11] Boehm, B. W. et al., Software Cost Estimation with COCOMO II. Prentice Hall, 2000.

[12] IFPUG, "Function Point Counting Practices Manual," Release 4.3.1, 2010.

[13] Papa, G. et al., "Machine Learning-Based Software Effort Estimation," IEEE Trans. Software Eng., 2020.

[14] Microsoft, "Azure Well-Architected Framework," Microsoft Docs, 2024.

[15] Liu, X. et al., "AgentBench: Evaluating LLMs as Agents," arXiv:2308.03688, 2023.

[16] Gao, Y. et al., "Retrieval-Augmented Generation for LLMs: A Survey," arXiv:2312.10997, 2023.